ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчет о программном проекте на тему:

Колоночный движок с нуля

(промежуточный, этап 1)

Выполнил студент:

группы #БПМИ243, 2 курса Бузин Игорь Тимурович

Принял руководитель проекта:

Перов Герман Сергеевич
Научный сотрудник
Факультет компьютерных наук НИУ ВШЭ

Москва 2026

Содержание

Аннотация **3**

1 Введение **4**

1.1 Описание предметной области 4

1.2 Постановка задачи 4

1.3 Структура работы 4

2 Обзор литературы **4**

3 Описание предлагаемого метода **5**

3.1 Структура классов 5

3.2 Колоночный формат bzn 6

4 Эксперименты **6**

5 Заключение **7**

Список литературы **8**

Аннотация

Цель проекта создать колоночный формат и колоночный движок который будет успешно проходить бенчмарк ClickBench.

Ключевые слова

Колоночный движок, колоночный формат, базы данных, ClickBench.

1 Введение

1.1 Описание предметной области

Колоночные движки являются популярным решением для исполнения SQL запросов к таблицам, которые не требуют функционала добавления/удаления строк. У них есть свой колоночный формат(теперь данные лежат столбцами, а не строками), что минимизирует количество cache miss'ов и чтений с диска при ответе на запрос.

Для оценки производительности движков предлагается использовать ClickBench. Это хороший способ оценить эффективность движков, бенчмарк прогоняет запросы на таблице, которая содержит порядка 100 столбцов и около 90 миллионов строк, при этом все данные анонимизированы и основаны на работе популярного сервиса, что делает условия максимально приближенными к реальным.

1.2 Постановка задачи

Нужно Пплучить таблицу hits.csv, создать свой колоночный формат на основе полученной таблицы, реализовать колоночный движок, работающий с новым форматом, который сможет успешно проходить бенчмарк ClickBench.

1.3 Структура работы

В рамках проекта был создан класс для чтения и записи csv файла а также класс для чтения и записи колоночного формата. Связь между ними осуществляется через класс batch (часть файла, которая помещается в оперативную память). Этот набор классов позволяет читать csv файл, преобразовывать его в колоночный формат и наоброт за пару простых шагов.

2 Обзор литературы

Так как мы нацелены именно на прохождение бенчмарка ClickBench, то посмотрим на лидеров по производительности на сайте бенчмарка [2]. Результаты запуском на сайте представлены на различных машинах, как с cold, так и warm режимами. Выберем тройку лидеров(все они являются колоночными). ClickHouse [3] Parquet [1] и DuckDB [4]. Выделим особенности каждого из движков.

ClickHouse [3] использует собственные форматы хранения (семейство движков MergeTree), сжатие и векторные инструкции процессора для обработки данных.

DuckDBduckdb – это встраиваемая (in-process) база данных. У неё нет сервера, она работает внутри процесса. Главная особенность – векторизованный движок выполнения запросов, который работает на одной машине.

Parquet [1] – это колоночный формат хранения данных, который содержит метаданные (статистику min/max), что позволяет движкам читать только нужные части файла (Predicate Pushdown).

У всех трёх движков есть свои особенности, которые позволяют им быть производительными.

3 Описание предлагаемого метода

3.1 Структура классов

class Schema - содержит информацию о названиях и типах столбцов таблицы.

class Column - содержит данные одного столбца таблицы, представляет из себя $std :: variant < std :: vector < type_1 >, ... >$ имеет метод $TranslateTo$ который переводит колонку из одного типа в другой.

class Batch - содержит часть таблицы, которая помещается в оперативную память, хранит данные как $std :: vector < Column >$ и держит рядом класс $Schema$, это сделано для того чтобы в батч можно было подставить другую схему, что вызовет трансформацию столбцов($TranslateTo$) в нужные типы.

class CSV-Reader - читает csv файл и возвращает batch строк согласно RFC-4180 [6].

class CSV-Writer - принимает batch дает новую схему в которой типы это строка, записывает в csv файл согласно RFC-4180[6]

class BZN-Reader - читает колоночный формат и возвращает batch

class BZN-Writer - принимает batch и записывает в колоночный формат (при инициализации создает массив с разметкой батчей, который заполняется по ходу записи)

Для более глубокого понимания всей структуры и методов можно обратится на GitHub [5]

3.2 Колоночный формат bzn

На данный момент поддерживается два типа данных: int64_t, string. Первый записывается как последовательность 8 байт в little-endian формате. Второй записывается как последовательность байт(символов), описывающая нашу строку, после чего идет `'\x1F'` – специальный символ, демонстрирующий конец строки.

Скажем, что количество колонок в нашей таблице это m и поговорим о роли batch'a в bzn формате: это набор колонок, соотвествующий какому-то количеству подряд идущих строк из таблицы, то есть вся исходная таблица представима в виде $batch_1 \sqcup batch_2 \sqcup ... \sqcup batch_k$, где k – количество batch'ей, зависящее от количества строк в таблице и размера batch'a(на данный момент размер определятеся как количество, хранящихся в batch'e).

batch в bzn формате лежит как: [метаданные][данные]. Метаданные хранятся как подряд идущие int64_t, которые описывают позиции в файле, с которых начинается каждая колонка $poscolumn_1, poscolumn_2, ..., poscolumn_m$. После метаинформации лежат сами данные колонок, с позиции $poscolumn_i$ в файле лежит i-ая колонка(о том как хрянятся различные типы уже говорили). Далее поговорим о метаданных всего файла. Из которых будет очевидно количество колонок в таблице, а значит и задача чтения батча очевидна.

Формат bzn выглядит следующим образом:
[размер метаданных][$batch_1$][$batch_2$]...[$batch_k$][метаданные] $batch_i$ хранится как сказано выше.

Метаданные в конце файла хранятся как:
[имена столбцов][типы столбцов][разметка батчей]

Где имена столбцов – это последовательность строк, разделенных символом `'\x1F'`, после которых идет символ `'\x1E'` говорящий об окончании этой секции метаданных. Типы столбцов хрянятся аналогично их строковыми представлениями. После них также идет символ `'\x1E'` и начинается разметка батчей, которая представляет собой последовательность int64_t, каждый из которых говорит о позиции в файле, с которой начинается соответствующий батч.

4 Эксперименты

На данный момент движок не реализован, поэтому экспериментальная часть состоит из перекладывания csv файла в колоночный формат и обратно с различными размерами batch'a(который определяется как количество строк).

Можно обратится к GitHub [5] и ознакомится с тестами, на которых проверяется кор-

ректность перевода из одного формата в другой. Таблицы из тестов. выглядят так:

Таблица 4.1: schema.csv схема из тестов к таблице ниже

a	int64
b	int64
name123	string
d	int64

Таблица 4.2: data.csv тестовая таблица для проверки перекладывания данных

1	2	first	4
5	1	second	2
8	17	third	2

На этих входных данных все корректно перекладывается(CI проверка GitHub прошла успешно).

5 Заключение

На данном этапе я могу создать колоночный формат из csv файла и обратно. Мне достаточно прочитать схему, создать CSVReader, читать из него батчи пока возможно, перекидывать их в MyFormatWriter и записывать в файл. Аналогично можно прочитать колоночный формат и записать его в csv файл. Далее предстоит создать сам движок, который будет работать с моим форматом и проходить бенчмарк ClickBench. Формат также может нуждаться в доработке, в силу отсутствия в нем каких-либо сжатий.

Список литературы

[1] *Apache Parquet information site*. URL: https://parquet.apache.org.

[2] *ClickHouse Benchmark results*. URL: https://benchmark.clickhouse.com.

[3] *ClickHouse information site*. URL: https://clickhouse.com.

[4] *DuckDB information site*. URL: https://duckdb.org.

[5] *GitHub repository project page*. URL: https://github.com/buziness11/columnar-engine/.

[6] *RFC 4180 specification of CSV format*. URL: https://www.rfc-editor.org/rfc/rfc4180.